

UNIVERSITATEA "ALEXANDRU-IOAN CUZA" DIN IAȘI

FACULTATEA DE INFORMATICĂ



LUCRARE DE LICENȚĂ

**Deeplyn - Platformă Web de suport psihologic
pentru persoanele cu tulburare de personalitate
borderline (BPD)**

propusă de

Nadia-Ioana Zapodeanu

Sesiunea: iulie, 2026

Coordonator științific

Conf. Dr. Ignat Anca

UNIVERSITATEA "ALEXANDRU-IOAN CUZA" DIN IAȘI

FACULTATEA DE INFORMATICĂ

**Deeplyn - Platformă Web de suport
psihologic pentru persoanele cu
tulburare de personalitate borderline
(BPD)**

Nadia-Ioana Zapodeanu

Sesiunea: iulie, 2026

Coordonator științific

Conf. Dr. Ignat Anca

Cuprins

Motivație	2
Introducere	3
1 Aplicații similare	4
1.1 DBT Coach	4
1.2 CBT Self-Care Journal	5
2 Tehnologiile utilizate	7
2.1 React	7
2.2 Tailwind	9
2.3 Axios	10
2.4 Django	10
2.5 Django REST-Framework	12
3 Clasificarea automată a emoțiilor	13
3.1 Detectarea automată a emoțiilor în limbaj natural	13
3.2 Transformatorul XLM-RoBERTa-base	14
3.3 Datele de antrenare pentru XLM-RoBERTa-base	16
3.4 Fine-tuning	20
3.5 Evaluarea modelului	22
4 Arhitectura platformei	26
4.1 Arhitectura generală a sistemului	26
4.2 Structura bazei de date	28
4.3 Interfața și implementarea platformei	30
4.3.1 Register și Login	30
4.3.2 Dashboard	32

4.3.3	Journal	34
4.3.4	DailyQuiz	35
4.3.5	Exercises	37
4.3.6	Therapist	39
5	 Direcții viitoare de dezvoltare	41
6	 Utilizarea uneltelor de Inteligență Artificială	42
	Concluzii	43
	Bibliografie	44

Motivație

Tulburarea de personalitate borderline (BPD)¹ influențează profund modul în care persoana se percepe pe sine și îi percepe pe cei din jur. Persoanele afectate pot interpreta situațiile și relațiile interpersonale prin reacții intense și instabilitate emoțională. Experiențele timpurii negative contribuie la dezvoltarea personalității și a percepțiilor despre mediul înconjurător al pacienților, fapt pentru care BPD este o tulburare cronică[1]. Aceste caracteristici contribuie la persistența simptomelor în timp și fac necesare ședințele de psihoterapie de durată. În cazurile mai severe, intervenția psihoterapeutică poate fi completată de sprijinul unui medic psihiatru, care poate recomanda tratament medicamentos pentru gestionarea simptomelor asociate.

Tocmai pentru că aceste persoane au o mare necesitate în a avea acces la terapie, am constatat că platformele dedicate tulburării sunt de obicei generice și incomplete. În cele mai multe cazuri, terapia în format fizic este inaccesibilă. În România, puțini psihoterapeuți sunt specializați în a oferi servicii de terapie potrivite tulburării, fapt pentru care pacienții sunt nevoiți să își găsească acest ajutor online.

Tema acestei lucrări am ales-o din dorința de a oferi o soluție software, prin care persoanele cu BPD să beneficieze de o platformă potrivită nevoilor lor. Am dezvoltat platforma **Deeplyn**, cu scopul de a oferi posibilitatea ca persoanele cu această tulburare să se conecteze cu un terapeut specializat, prin care să pun la dispoziție instrumente ce asigură o continuitate între ședințe.

Am conceput platforma astfel încât să fie flexibilă și ușor de utilizat. Utilizatorii pot folosi instrumentele disponibile independent sau în cadrul unei relații terapeutice active.

¹abreviere provenită din limba engleză pentru Borderline Personality Disorder

Introducere

Deeplyn este o platformă web pe care am dezvoltat-o pentru persoanele diagnosticate cu tulburare de personalitate borderline, dar și pentru oricine vrea să-și înțeleagă mai bine emoțiile. Sursa principală de inspirație în implementarea exercițiilor propuse este *terapia dialectic-comportamentală (DBT)*² și *terapia cognitiv-comportamentală (CBT)*³, două abordări validate clinic în psihoterapie. Am optat să integrez aceste două metode deoarece sunt cele mai studiate în raport cu BPD. Principiile acestora pot fi relevante pentru oricine vrea să înțeleagă mai bine de unde vin anumite reacții, să își modifice convingeri despre sine sau să-și regleze mai eficient stările emoționale.

Platforma a fost gândită să fie disponibilă indiferent dacă utilizatorul are sau nu un terapeut activ, astfel că majoritatea componentelor aplicației pot fi folosite în oricare dintre cele două situații. Printr-o secțiune dedicată, am oferit posibilitatea persoanelor interesate să ia legătura cu psihoterapeuți specializați direct din aplicație.

Deeplyn se bazează pe patru funcționalități principale și anume:

- Un jurnal pentru gânduri și emoții, pe care utilizatorul îl poate asocia cu evenimente concrete din viața lui. Fiecare notiță scrisă poate fi etichetată manual sau automat, cu ajutorul unui model de tip transformer ajustat (fine-tuned⁴), și poate fi trimisă direct terapeutului din aplicație.
- Daily Quiz, un chestionar zilnic cu întrebări de reflecție, creat pentru a completa sesiunile de psihoterapie și pentru a facilita procesul de autocunoaștere.
- Exerciții din DBT și CBT pentru reglarea emoțională și restructurarea gândurilor.
- O secțiune de progres cu statistici despre stările emotionale pe parcursul unei săptămâni, luni sau ani.

²Abreviere din limba engleză, Dialectical Behavior Therapy

³Abreviere din limba engleză, Cognitive Behavioral Therapy

⁴Procesul prin care un model pre-antrenat este reantrenat pe un nou set de date pentru a fi adaptat unei sarcini specifice

Capitolul 1

Aplicații similare

Înainte de a defini funcționalitățile platformei **Deeplyn**, am explorat câteva aplicații mobile similare. Le-am parcurs ca utilizator obișnuit, ceea ce mi-a permis să observ ce funcționează în practică, unde apar frustrările și ce lipsește în mod recurent. Majoritatea ofereau exerciții din DBT sau CBT, un spațiu de reflecție sau, uneori, un jurnal zilnic pentru monitorizarea stărilor emoționale. Am constatat așadar că puține reușeau să combine toate acestea într-o singură platformă.

1.1 DBT Coach

*DBT Coach*¹ este dezvoltată de Resiliens pentru persoanele cu BPD, anxietate, depresie, ADHD sau alte afecțiuni de sănătate mintală.

Structura aplicației este orientată mai mult pe exercițiile specifice DBT, unde persoanele pot exersa abilități noi, pe care le pot practica în viața de zi cu zi. Platforma este disponibilă atât pe iOS, cât și pe Android, ceea ce o face accesibilă indiferent de dispozitiv.

Acest instrument digital vine cu o serie de funcționalități foarte bine structurate. Este inclus un jurnal zilnic în care persoanele își notează starea emoțională, tendințele specifice BPD și skill-urile învățate de pe urma activităților propuse. Ce mi s-a părut interesant este că înregistrările, chiar dacă la prima vedere par un formular clasic, de fapt îi ajută în timp pe useri să își recunoască propriile tipare de gândire.

O altă componentă implementată de ei este un chatbot, prin care utilizatorii pot comunica deschis despre ce simt, cum au reacționat și ce a declanșat trăirea respec-

¹<https://apps.apple.com/us/app/dbt-coach/id1452264969>

tivă. Mi s-a părut cea mai utilă funcționalitate mai ales în situațiile de criză, deoarece răspunde empatic și înțelegător la confesiunile userului. Tot el recomandă exerciții adaptate stării identificate din conversație și îl încurajează să le aplice pentru a diminua intensitatea trăirilor.

Un minus pe care l-am observat este că o bună parte din conținut nu e gratuit. Multe componente esențiale, precum tehnicile sugerate, conectarea cu terapeutul și cursurile, sunt marcate ca premium, ceea ce nu e deloc evident la prima vedere. Abia când încerci să le accesezi îți dai seama că trebuie să plătești un abonament de minim 59,99 de lei pe lună pentru a le vizualiza. Practic, exercițiile DBT din aplicație sunt, în mare parte, după un paywall.

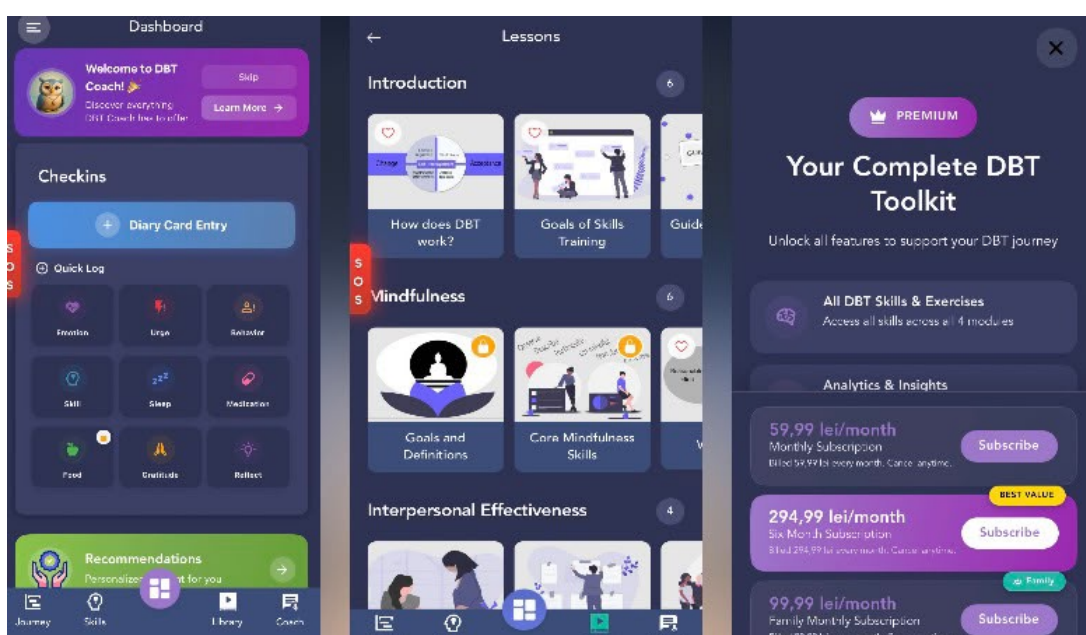


Figura 1.1: Capturi de ecran din aplicația DBT Coach

1.2 CBT Self-Care Journal

*CBT Self-Care Journal*² este o aplicație iOS dezvoltată de MindMatters, destinată publicului larg, în special persoanelor care vor să își reconstruiască convingeri mai sănătoase sau să își schimbe perspectiva asupra situațiilor care le-au generat tipare de gândire negative.

Aplicația oferă exerciții din CBT în care cei interesați sunt ghidați prin scenarii concrete, menite să îi ajute să identifice momentele în care și-au format convingeri

²<https://apps.apple.com/us/app/cbt-self-care-journal/id1523102810>

limitative. Ca structură, se aseamănă cu tool-ul descris în secțiunea anterioară.

Cei înregistrați sunt încurajați zilnic să scrie în jurnal și să parcurgă activitățile propuse, soluția fiind poziționată în clasa instrumentelor de dezvoltare personală independentă, fără implicarea unui specialist.

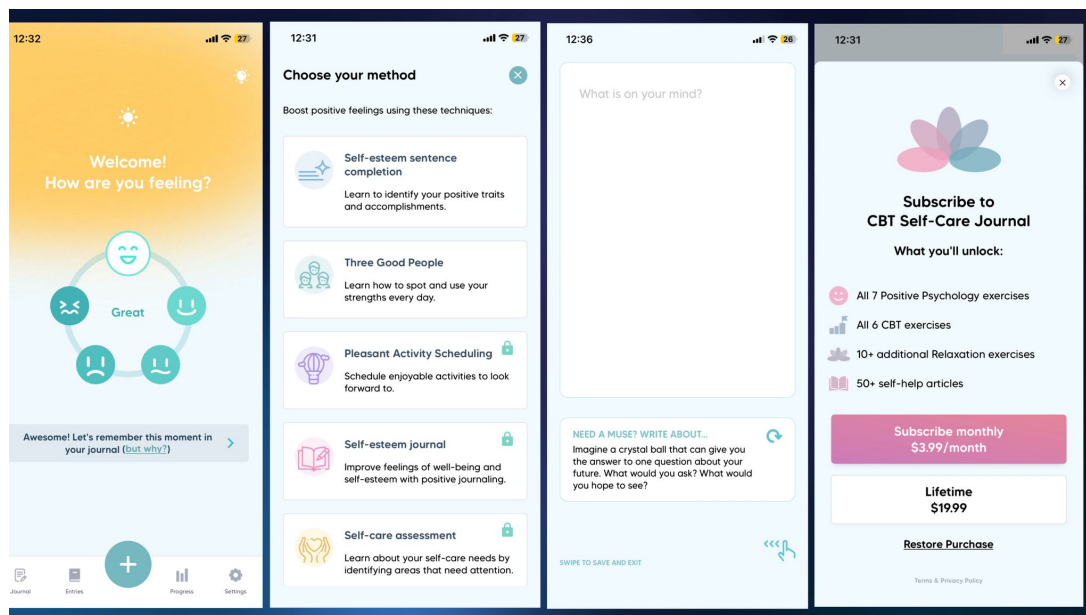


Figura 1.2: Capturi de ecran din aplicația CBT Self-Care Journal

Am observat totuși câteva limitări. Aplicația nu include o opțiune de conectare cu un specialist, ceea ce îi conferă un caracter relativ static; sarcinile sunt predefinite și nu se ajustează în funcție de nevoile fiecărui utilizator.

Ca și în cazul platformei descrise anterior, accesul complet la cursuri și exerciții necesită un abonament lunar mai accesibil ca preț, însă prezent. Dincolo de aspectul financiar, aplicația oferă un număr destul de limitat de resurse, ceea ce o face mai potrivită pentru persoanele aflate într-un stadiu incipient, care vor să exploreze tehnicile CBT pe cont propriu.

Un aspect pe care l-am considerat important de menționat este absența oricărui mecanism de urgență. Platforma nu oferă persoanei nicio modalitate de a contacta un terapeut sau un serviciu de intervenție în situațiile critice.

Consider că terapia ar trebui să fie accesibilă unui public cât mai larg, mai ales persoanelor care vor să lucreze la propriile emoții fără a apela neapărat la un terapeut. Tocmai de aceea, în *Deeplyn* am urmărit să ofer gratuit resurse și exerciții inspirate din ambele metode validate clinic, fără restricții de abonament.

Capitolul 2

Tehnologiile utilizate

2.1 React

React[2](cunoscut și sub numele de React.js sau ReactJS) este o bibliotecă open-source dezvoltată de Meta, pentru construirea interfețelor dinamice și interactive ale aplicațiilor web, fiind responsabilă de partea vizuală cu care utilizatorul interacționează. Spre deosebire de un framework complet, React este o bibliotecă ce oferă flexibilitate în implementare.

Interfețele pot fi construite din componente reutilizabile și independente, precum se poate observa în Figura 2.1. Odată implementată, o componentă poate fi apelată de câte ori e nevoie, aspect ce elimină redundanța codului. React încurajează principiul responsabilității unice, util mai ales când proiectul crește și codul trebuie întreținut pe termen lung. În plus, această bibliotecă oferă cât mai puține constrângeri în dezvoltarea oricărui proiect, caracteristică ce îl diferențiază de alternative precum Angular sau Vue.js.

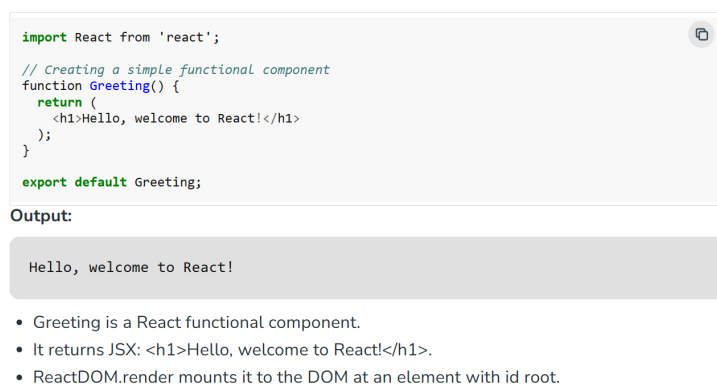


Figura 2.1: Sintaxa unei componente React

Pentru procesul de randare a tuturor componentelor interfeței, React utilizează mecanismul Virtual DOM (Document Object Model), ce presupune o copie virtuală a paginii web păstrată în memorie. De fiecare dată când apare o modificare, Virtual DOM-ul este comparat cu starea anterioară a paginii, iar actualizările sunt aplicate doar asupra elementelor care s-au schimbat. Prin acest mecanism, React este considerat optim, utilizarea lui facilitează tot procesul de creare a unei aplicații rapide, fluide și ușor de întreținut.[2]

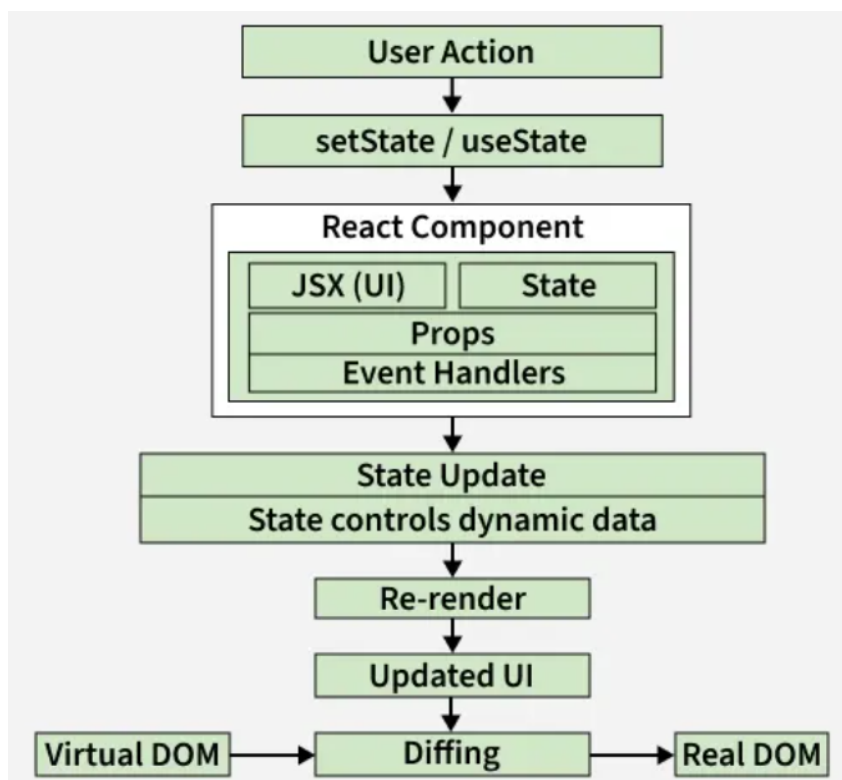


Figura 2.2: Randarea tuturor componentelor¹

Am ales să utilizez React în cadrul acestei lucrări deoarece, spre deosebire de un framework complet, React nu impune o structură fixă a proiectului și nici o arhitectură bazată pe reguli stricte. Acest lucru mi-a oferit libertatea de a proiecta singură arhitectura aplicației și de a mă concentra asupra dezvoltării componentelor vizuale și a logicii asociate acestora.

Un alt motiv care a influențat această alegere a fost experiența acumulată în timpul facultății. Am utilizat React în cadrul disciplinei „Ingineria Programării”, iar familiaritatea cu această tehnologie a facilitat dezvoltarea aplicației și aprofundarea cunoștințelor dobândite anterior.

¹Figurile 2.1 și 2.2 preluate din: <https://www.geeksforgeeks.org/reactjs/reactjs-components/>

În plus, popularitatea React atât în rândul dezvoltatorilor independenți, cât și al companiilor mari, a reprezentat un argument suplimentar în alegerea acestei biblioteci pentru implementarea părții de frontend a aplicației Deeplyn. Conform Stack Overflow Developer Survey 2025[3], React este utilizat de 44,7% dintre programatori la nivel global, depășind alternative precum Angular (18,2%) și Vue.js (17,6%), un indicator clar că ecosistemul e matur și că suportul comunității nu dispare prea curând, după cum se poate observa în Figura 2.3.

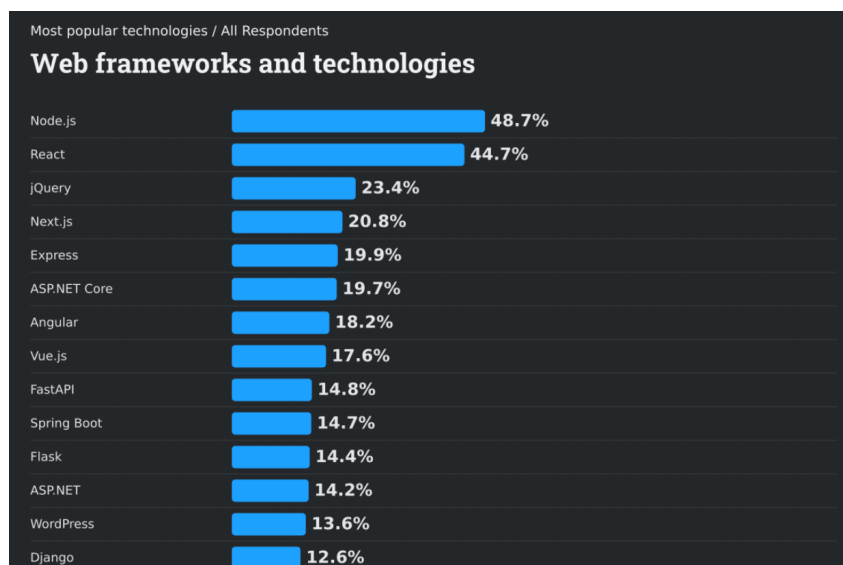


Figura 2.3: Cele mai utilizate tehnologii web în 2025[3]

2.2 Tailwind

Tailwind CSS este un framework modern utilizat pentru stilizarea interfețelor web. Această tehnologie a fost adoptată de multe companii populare la nivel mondial, precum NASA, Netflix sau Shopify, fapt ce demonstrează ideea că framework-ul poate susține aplicații de mari dimensiuni.

Am selectat acest framework pentru aplicația Deeplyn, în principal deoarece stilizarea se aplică direct în fișierele .jsx. Sistemul de breakpoint-uri a reprezentat un factor la fel de important în luarea acestei decizii; prefixele sm:, md:, lg: se aplică direct pe elemente, eliminând necesitatea scrierii unor blocuri separate de media queries. Interfața grafică și experiența utilizatorului (UI/UX) au constituit priorități majore în cadrul proiectului, urmărindu-se asigurarea unei accesibilități optime pe toate tipurile de ecrane. În plus, utilizarea claselor de utilitate direct pe tag-urile HTML a facilitat

construirea interfeței, astfel încât, spre deosebire de CSS-ul clasic, Tailwind a optimizat substanțial procesul de dezvoltare pe partea de frontend.

2.3 Axios

Axios este o bibliotecă JavaScript utilizată pentru comunicarea dintre frontend și server prin intermediul cererilor HTTP. Comparativ cu fetch, soluția nativă oferită de browser, Axios simplifică procesul de lucru prin conversia automată a datelor în format JSON și prin posibilitatea configurării unei instanțe reutilizabile pentru întregul proiect.

În cadrul aplicației Deeplyn, am configurat o instanță personalizată de Axios care atașează automat token-ul CSRF fiecărei cereri și gestionează sesiunea utilizatorului. În cazul în care sesiunea expiră, reînnoirea tokenului se face automat în fundal, fără intervenția utilizatorului, iar dacă aceasta eșuează, utilizatorul este redirecționat către pagina de autentificare.

2.4 Django

Django[4] este un framework open-source scris în Python, utilizat pentru implementarea logicii aplicațiilor web și, în special, pentru dezvoltarea componentei de backend. Definirea tabelelor bazei de date este posibilă prin scrierea lor în clase Python, acest aspect oferind o soluție pentru simplificarea dezvoltării aplicațiilor complexe.

Django reduce considerabil timpul necesar implementării funcționalităților uzuale, precum autentificarea utilizatorilor, administrarea conținutului sau gestionarea sesiunilor. E un framework popular datorită structurii sale solide, dar și pentru că ecosistemul Python e bine întreținut și are o comunitate activă.

Modelul arhitectural Model-View-Template (MVT), care separă logica aplicației de interfața utilizatorului este prezent în Django. Fiecare componentă are un rol clar, ceea ce face codul mai organizat și mai ușor de urmărit.

Față de alte framework-uri de backend, Django se distinge prin câteva caracteristici native care simplifică organizarea și administrarea proiectului.

Una dintre ele este sistemul de aplicații reutilizabile numite "apps", ce lucrează independent, fiecare cu propriile modele, view-uri și URL-uri. Avantajul lor este că pot fi extrase și integrate în alte proiecte, fără modificări majore. În Deeplyn, de exemplu,

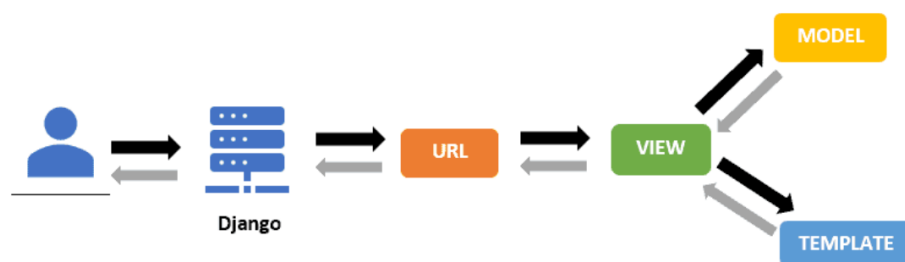


Figura 2.4: Arhitectura MVT

am implementat logica pentru sesiuni, jurnale, quizuri și exerciții în aplicații separate, ceea ce a făcut dezvoltarea platformei mai ușor de urmărit și de testat independent.

Panoul de admin este generat automat, fiind o caracteristică specifică Django. Cu alte cuvinte, în cazul în care dorești să dezvolti rapid o aplicație CRUD funcțională în browser, poți să definești o bază de date și să înregistrezi câteva linii în `admin.py`. Această parte de admin m-a ajutat considerabil în implementarea platformei mele și mi-a oferit acces mai facil la baza de date, fără să construiesc unelte separate.

ORM introduce ideea că modelele din aplicație sunt reprezentate prin clase Python, fiecare clasă corespunzând unui tabel din baza de date. Django generează automat interogările SQL și le parametrizează, ceea ce reduce riscul vulnerabilităților de tip SQL Injection² și simplifică procesul de dezvoltare. Această particularitate reprezintă un avantaj pentru dezvoltatorii care își doresc să își structureze o parte considerabilă a proiectului, utilizând un singur limbaj de programare.[4]

Pe partea de securitate, Django oferă protecție împotriva atacurilor XSS (Cross-Site Scripting) și CSRF (Cross-Site Request Forgery). Parolele sunt stocate prin algoritmi moderni de hashing, precum PBKDF2, iar sesiunile sunt gestionate prin cookie-uri securizate.

Implicit, Django folosește SQLite3, un sistem de gestiune a bazelor de date relaționale, care rulează pe același server concomitent. SQLite3 e potrivit pentru aplicații cu trafic redus și volume mici de date, tocmai pentru că nu consumă multe resurse.

Dezavantajul major al acestei bazei de date o reprezintă scalabilitatea. Pentru aplicațiile complexe unde traficul de date și dimensiunea numărului de utilizatori este mare, SQLite3 întâmpină dificultăți la operațiile concurente, care pot bloca temporar baza de date, timpul de răspuns amplificându-se.

²O vulnerabilitate în securitatea dezvoltării platformelor web prin care un atacator introduce cod malițios în câmpurile de input aflate în interfața utilizatorilor.

Am ales Django pentru implementarea backend-ului platformei *Deeplyn* atât pentru caracteristicile tehnice prezentate mai sus, cât și pentru faptul că este un framework adoptat pe scară largă în industrie. Printre companiile care îl utilizează se numără Instagram, Pinterest și Mozilla, ceea ce confirmă că poate susține aplicații complexe.

2.5 Django REST-Framework

Django REST Framework (DRF) este o bibliotecă construită peste Django, care simplifică dezvoltarea API-urilor REST.

DRF oferă componente predefinite care acoperă tot procesul de construire al unui endpoint și scurtează considerabil din munca de implementare. În Deeplyn, toată comunicarea dintre frontend-ul React și backend-ul Django trece prin DRF, de la autentificarea utilizatorilor până la gestionarea jurnalelor și a sesiunilor de terapie.

Capitolul 3

Clasificarea automată a emoțiilor

3.1 Detectarea automată a emoțiilor în limbaj natural

Recunoașterea și verbalizarea emoțiilor sunt importante în procesul terapeutic. Identificând stările afective înainte de a reacționa la o situație, putem conștientiza existența unui *pattern* nociv și, ulterior, să îl schimbăm. Persoanele cu borderline au tendința de a avea o dificultate în recunoașterea propriilor emoții, conform studiului Flasbeck et al. [5] ce aduce în prim-plan tocmai acest aspect. Astfel, nouăsprezece persoane diagnosticate cu tulburare borderline, respectiv optsprezece persoane sănătoase, au luat parte în cadrul cercetării, fiind separate în două grupuri.

Participanților li s-au atașat electrozi pe cap și pe piept, pentru a determina reactivitatea și intensitatea electrică a creierului în concordanță cu bătăile inimii. A fost observată o diferență majoră între cele două grupuri. Activitatea frontală a participanților cu BPD era mult mai ridicată decât cea a grupului aferent[5]. În medicină, acest fapt se traduce printr-o *suprasolicitare* care compromite capacitatea de interpretare a semnalelor corporale. O analogie similară cu această situație este volumul ridicat al radioului: chiar dacă este la maximum, nu înseamnă și că vom înțelege mesajul transmis de acesta.

Am decis să introduc în platforma Deeplyn o funcționalitate care să sprijine pacienții cu BPD în identificarea propriilor emoții. Am ales să ofer flexibilitate în utilizarea acesteia și să încurajez utilizatorii să își identifice trăirile resimțite, specific psihoterapiei.

În componentele dedicate scrierii libere, am urmărit să ofer flexibilitate în procesul de etichetare a emoțiilor. Astfel, utilizatorii pot alege ca textele redactate, precum notițele din jurnal, să nu fie asociate unei emoții, să fie etichetate manual sau să benefi-

cieze de etichetare automată. Pentru această din urmă opțiune, am integrat un model de tip Transformer, pe care l-am optimizat și specializat pentru identificarea primelor trei emoții predominante din textele introduse de utilizatori.

3.2 Transformatorul XLM-RoBERTa-base

XLM-RoBERTa-base este un model propus de cei de la Meta AI, ce reprezintă o extensie a transformatorului **RoBERTa**, și este conceput să înțeleagă conținutul textelor redactate în limbi diferite. Pentru procesarea și interpretarea textelor ample, transformatorul are ca bază *mecanismul de atenție* (self-attention), ce funcționează similar cu modelele precedente.

Inputul textual este împărțit în cuvinte și fragmente mici ale acestora, ce li se atribuie câte un ID numeric unic din vocabularul prestabilit de dezvoltatori. Vocabularul este comun tuturor limbilor pe care XLM-RoBERTa a fost pre-antrenat, construit prin intermediul algoritmului SentencePiece. Acest prim proces este cunoscut ca *tokenizarea* datelor de intrare.

După procesul de tokenizare, fiecărui token i se atribuie un *embedding*. Pentru transformator, importanța cunoașterii poziției tokenului este capitală deoarece, prin acest mod, poate să înțeleagă mesajul textului. Dacă nu am reține această poziție, exemplul "mă bucur că nu sunt trist" poate fi înțeles și ca "nu mă bucur că sunt trist", iar sensul mesajului să fie complet degradat.

Diferența substanțială dintre rețelele neuronale recurente și transformatori o constă mecanismul de atenție menționat anterior. Dacă în trecut, aceste rețelele neuronale, precum LSTM (Long Short-Term Memory) sau GRU (Gated Recurrent Unit), pierdeau înțelesul textelor, transformatorii reușesc să țină cont de subtilitățile lingvistice ale acestora prin intermediul algoritmului de *self-attention*. Algoritmul propune trei matrici învățate în antrenare, **Q**, **K**, **V**, fiecare având rol separat.

Matricea Q (query) este utilizată pentru fiecare token din secvență și interoghează toți ceilalți tokeni dacă există în propoziție elemente care pot modifica sensul.

Matricea K(key) este responsabilă în a prezenta scorurile de similitudine între tokeni. Acest număr ne ajută să înțelegem relațiile dintre fragmentele de cuvinte, cu cât scorul este mai mare, cu atât legătura semantică este mai strânsă și poate schimba sensul textual.

Matricea V(value) determină informația finală, respectiv sensul cuvintelor. Dacă

observăm o potrivire semnificativă între matricea Q și matricea K, tokenul interogat preia informația din matricea V.

Voi lua ca exemplu relevant, în special în detectarea emoțiilor, propoziția "mă simt nefericită astăzi". Când textul este tokenizat, la un moment dat transformatorul ajunge să analizeze tokenul "fericit". Observăm că primul calcul îl face pe matricea Q care specifică existența în propoziție a unor elemente care pot modifica sensul acestui token. Matricea K a tokenului "ne" returnează un scor de similitudine ridicat cu matricea Q a lui "fericit". La final, matricea V transmite informația că sensul tokenului "fericit" este negat. Tocmai această separare între cele trei matrici face self-attention flexibil și optim.

Acest algoritm îl aplicăm de douăsprezece ori în paralel, pe câte un *attention head*. Orice attention head are propriile matrici Q, K, V independente pentru ca modelul să învețe și să detecteze tipuri diferite de relații. În final, rezultatul celor douăsprezece capete se concatenează pentru a forma o reprezentare unificată. Conform Vaswani et al. [6], calculăm acest self-attention pentru toți tokenii secvenței prin formula:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (3.1)$$

unde $Q \in \mathbb{R}^{n \times d_k}$, $K \in \mathbb{R}^{n \times d_k}$ și $V \in \mathbb{R}^{n \times d_v}$ sunt matricile query, key și value menționate anterior, n reprezintă lungimea secvenței de tokeni din input, anume textul inițial după tokenizare, iar $d_k=64$ este dimensiunea unui attention-head. Numărul 64 provine din împărțirea dimensiunii hidden la cele 12 capete de atenție ($768/12 = 64$). Împărțirea la $\sqrt{d_k}$ ne ajută să scalăm scorurile pentru a avea o stabilizare numerică înainte de funcția softmax.

3.3 Datele de antrenare pentru XLM-RoBERTa-base

În alegerea datasetului corespunzător platformei Deeplyn, primul analizat a fost GoEmotions, disponibil pe platforma Hugging Face. Acesta cuprinde texte preluate de pe Reddit, etichetate cu 28 de emoții [7], fapt ce părea în prima fază o alegere inedită. După o analiză amănunțită, am observat lacunele acestui set de date, anume faptul că distribuția exemplelor per emoție este profund dezechilibrată, precum se poate observa în Figura 3.1.

total exemple train: 51103									
total exemple val: 6380									
total exemple test: 6329									
total: 63812									
emotion	train	val	test	total	sadness	1326	143	156	1625
neutral	14219	1766	1787	17772	disappointment	1269	163	151	1583
admiration	4130	488	504	5122	realization	1110	127	145	1382
approval	2939	397	351	3687	caring	1087	153	135	1375
gratitude	2662	358	352	3372	surprise	1060	129	141	1330
annoyance	2470	303	320	3093	excitement	853	96	103	1052
amusement	2328	303	264	2895	disgust	793	97	123	1013
curiosity	2191	248	284	2723	desire	641	77	83	801
love	2086	252	238	2576	fear	596	90	78	764
disapproval	2022	292	267	2581	remorse	545	68	56	669
optimism	1581	209	186	1976	embarrassment	303	35	37	375
anger	1567	195	198	1960	nervousness	164	21	23	208
joy	1452	172	161	1785	relief	153	18	11	182
confusion	1368	152	153	1673	pride	111	15	16	142
					grief	77	13	6	96

Figura 3.1: Distribuția exemplelor per emoții al datasetului GoEmotions[7]

Cum publicul țintă al aplicației Deeplyn îl reprezintă persoanele care necesită suport emoțional digital, majoritatea dataseturilor identificate, care sunt open-source, nu satisfac nevoile de bază ale utilizatorilor. Emoțiile acoperă un spectru larg de categorii. Multe dataseturi prezintă dezavantaje asemenea celui prezentat anterior, anume exemplele clasificate nu acoperă un registru suficient de larg de stări emoționale sau datele nu sunt distribuite echilibrat.

Din acest motiv, am selectat mai multe seturi de date și le-am concatenat, pentru a optimiza modelul să răspundă cu o acuratețe cât mai mare în detectarea emoțiilor predominante dintr-un text.

Pentru antrenarea modelului, am luat decizia de a uni cinci seturi de date complementare, deoarece am dorit să acopăr un spectru larg de emoții și să obțin o distribuție cât mai echilibrată a exemplelor per clasă.

Primul dataset pe care l-am luat în calcul este ISEAR (International Survey on Emotion Antecedents and Reactions) [8] (figura 3.2), cunoscut pentru aplicabilitatea în platformele din industrie pentru procesarea limbajului natural și recunoașterea emoțiilor la nivel textual.

```
print("inainte de deduplicare:", len(df))

df = df.drop_duplicates(subset=["SIT"])

print("dupa deduplicare:", len(df))
print("duplicate gasite:", len(df) - len(df))
print("\n distributie label-uri:")
print(df["Field1"].value_counts())
```

inainte de deduplicare: 5874
dupa deduplicare: 5845
duplicate gasite: 0

distributie label-uri:

Field1	
anger	854
disgust	853
fear	850
joy	836
guilt	824
sadness	816
shame	812

Name: count, dtype: int64

Figura 3.2: Distribuția exemplelor unice per emoții al datasetului ISEAR[8]

L-am luat în considerare deoarece conține aproximativ șase mii de texte clasificate prin intermediul celor 7 emoții de bază. Toate aceste texte au fost colectate la nivel internațional, unde participanților li s-a cerut să descrie o situație reală din viața personală, în care au experimentat una din emoțiile propuse. Am optat ca acesta să fie inclus în antrenarea modelului deoarece se apropie semnificativ de notițele din jurnal; utilizatorii cu BPD sunt predispuși în a scrie lucruri intime, personale și pline de emoție, în special în situațiile coplesitoare.

Al doilea dataset a fost preluat de pe platforma Hugging Face și este o versiune reinterpretată a datasetului EmpatheticDialogues [9], ce conține o colecție de texte narrative la persoana I, clasificate pe 32 de clase de emoții (figura 3.3). Participanții acestui set de date descriu situații personale asociate unei trăiri specifice. Această caracteristică face ca datele să fie compatibile cu stilul notițelor din jurnal, relevante pentru utilizatorii Deeplyn.

Un alt corpus inclus în antrenare este CARER [10], reprezentat în figura 3.4, ale cărui exemple provin din rețeaua de socializare Twitter. Conține mesaje scurte, une-

```

ex ramase: 19197

distributie label-uri dupa curatare:
emotion
surprised      997    confident      609
excited        731    anticipating    597
angry          676    nostalgic      595
proud          665    joyful         595
sad            659    disappointed    593
annoyed        656    furious        592
grateful       637    prepared       584
lonely         633    jealous        579
afraid         618    content        568
hopeful        615    embarrassed    560
terrified      612    devastated     553
disgusted      611    sentimental    512
guilty         611    caring         504
impressed      611    trusting       497
anxious        611    ashamed        485
               611    apprehensive   460
               611    faithful       371
Name: count, dtype: int64

```

Figura 3.3: Distribuția exemplilor unice per emoții al datasetului EmpatheticDialogues[9]

ori lipsite de subiect explicit, și reflectă o formulare compactă, specifică rețelelor de socializare. Datasetul acoperă șase emoții, printre care se enumeră furia, bucuria, iubirea, tristețea și surpriza. Am ales să includ și acest model pentru ca modelul de tip transformer să fie optim și asupra textelor scurte, nu numai asupra textelor ample.

```

from datasets import ClassLabel

ds_dair = load_dataset("dair-ai/emotion")
df_dair = pd.DataFrame(ds_dair["train"])

print("coloane:", df_dair.columns.tolist())
print("\n nr total exemple:", len(df_dair))
print("\n label-uri unice:")
print(df_dair["label"].value_counts())

features = ds_dair["train"].features
print(features["label"].names)

coloane: ['text', 'label']

nr total exemple: 16000

label-uri unice:
label
1    5362
0    4666
3    2159
4    1937
2    1304
5     572
Name: count, dtype: int64
['sadness', 'joy', 'love', 'anger', 'fear', 'surprise']

```

Figura 3.4: Distribuția exemplilor unice per emoții al datasetului CARER(dair-ai)[10]

Setul de date care a acoperit lacunele distribuției textelor per clasă de emoții este setul cu 20 de emoții [11] (figura 3.5), ce a venit în completarea celorlalte surse utilizate. Emoțiile distincte precum gelozia, recunoștința, confuzia sau liniștea erau absente și insuficient reprezentate în exemplele colectate până în acel punct.

```
from datasets import ClassLabel

ds_dair = load_dataset("dair-ai/emotion")
df_dair = pd.DataFrame(ds_dair["train"])

print("coloane:", df_dair.columns.tolist())
print("\n nr total exemple:", len(df_dair))
print("\n label-uri unice:")
print(df_dair["label"].value_counts())

features = ds_dair["train"].features
print(features["label"].names)
```

```
coloane: ['text', 'label']

nr total exemple: 16000

label-uri unice:
label
1      5362
0      4666
3      2159
4      1937
2      1304
5       572
Name: count, dtype: int64
['sadness', 'joy', 'love', 'anger', 'fear', 'surprise']
```

Figura 3.5: Distribuția exemplelor unice per emoții al datasetului cu cele 20 de emoții[11]

label_mapped			
anger	5000	love	5000
disgust	5000	joy	5000
embarrassment	5000	hope	4977
surprise	5000	jealousy	4657
fear	5000	gratitude	4640
happiness	5000	disappointment	4415
guilt	5000	loneliness	4371
sadness	5000	excitement	4369
pride	5000	neutral	3997
		relief	3963
		Name: count, dtype: int64	

Figura 3.6: Distribuția exemplor unice per emoții al datasetului final

Dataseturile menționate anterior cuprind exemple etichetate cu toate emoțiile esențiale antrenării. Însă, uneori putem scrie și despre o situație care nu prezintă o emoție clară. Ultima emoție vizată pe care am decis să o includ în datasetul complet este "neutral". Cum niciun set de date din cele menționate mai sus nu au cuprins și această emoție, am preluat aproximativ patru mii de exemple alese aleatoriu din datasetul GoEmotions[7].

Înainte de concatenarea tuturor dataseturilor, am verificat absența duplicatelor, deoarece exemplele repetate pot înrăutăți antrenarea, iar modelul poate memora datele de antrenament (*overfitting*). În ilustrația 3.6, este prezentată distribuția finală a tuturor emoțiilor pe texte unice, rezultate din concatenarea dataseturilor menționate anterior.

3.4 Fine-tuning

Setul de date creat, cu care vom lucra în continuare, este compus din 90.389 de exemple. Înainte de a antrena modelul XLM-RoBERTa-base, am împărțit setul de date în trei componente: 80% din exemple reprezintă textele pentru antrenare (train), 10% sunt datele de validare (validation), iar restul de 10% au fost alocate pentru testare (test). Împărțirea a fost realizată în funcție de clasa de emoție, astfel încât fiecare subset să păstreze aceeași distribuție a celor 19 emoții din setul original. Această divizare a datelor oferă o evaluare realistă a capacității de generalizare a modelului.

Primul pas al antrenării constă în *tokenizarea* tuturor exemplurilor din setul de date. Pentru această etapă, am folosit *tokenizatorul* nativ al modelului XLM-RoBERTa-base, stabilind o lungime maximă de 128 de tokeni pentru fiecare text. Pentru textele mai scurte, care nu atingeau acest prag, au fost adăugați tokeni speciali de completare [PAD], astfel încât fiecare secvență să aibă aceeași lungime.

Am optat pentru această valoare deoarece exemplele din setul de date au o dimensiune mică, majoritatea încadrându-se în limita de 128 de tokeni. Dacă aş fi ales o valoare mai mare, precum 256 sau 512 tokeni, aceasta nu ar fi îmbunătățit performanța în mod real. Mecanismul de atenție ar fi avut o creștere pătratică în raport cu lungimea secvenței la momentul calculului, ceea ce ar fi dus la un consum mult mai mare de memorie.

Pe lângă procesarea textului, funcția din Figura 3.7 se ocupă și de transformarea fiecărei etichete (`label_id`) într-un vector binar prin tehnica *onehot encoding*. Fiecare identificator de clasă (`label_id`) este convertit într-un vector de dimensiune 19, ce conține valori de 0.0 și 1.0, unde doar poziția corespunzătoare etichetei curente primește valoarea 1.0.

```
def tokenize_and_encode(examples):
    tokens = tokenizer(
        examples["text"],
        max_length=128,
        truncation=True,
        padding="max_length")

    # transformare label_id in vector de 19 pozitii pentru multi-label -->>> sigmoid
    labels = []
    for l_id in examples["label_id"]:
        one_hot = [0.0] * num_classes
        one_hot[l_id] = 1.0
        labels.append(one_hot)

    tokens["labels"] = labels
    return tokens

# sterg coloanele vechi
coloane_vechi = dataset["train"].column_names
tokenized_dataset = dataset.map(tokenize_and_encode, batched=True, remove_columns=coloane_vechi)
print("coloane ramase:", tokenized_dataset["train"].column_names)
```

Figura 3.7: Transformarea etichetelor în vectori binari pentru antrenare

În practică, în aplicația Deeplyn, utilizatorii vor scrie texte lungi. Din acestea putem să extragem în mod natural mai multe emoții care pot exista în același timp, în mod independent. Prin metoda `dataset.map` aplicată în grupuri (`batched=True`), vechile coloane sunt eliminate pentru a păstra doar reprezentările tokenizate și vectorii binari ce reprezintă etichetele menționate anterior.

Tot acest proces de antrenare a fost realizat pe platforma Kaggle, unde am utilizat două GPU-uri T4 în paralel. Pentru reducerea consumului de memorie și pentru accelerarea calculelor, am activat precizia mixtă prin parametrul `fp16=True`.

```
from transformers import TrainingArguments, Trainer, EarlyStoppingCallback

training_args = TrainingArguments(
    output_dir="./emotion_model_results",
    learning_rate=2e-5,
    per_device_train_batch_size=32,
    per_device_eval_batch_size=32,
    num_train_epochs=10,
    weight_decay=0.01,
    eval_strategy="epoch",
    save_strategy="epoch",
    fp16=True,
    load_best_model_at_end=True,
    metric_for_best_model="macro_roc_auc",
    logging_steps=100,
    report_to="none")

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset["train"],
    eval_dataset=tokenized_dataset["validation"],
    compute_metrics=compute_metrics,
    callbacks=[EarlyStoppingCallback(early_stopping_patience=2, early_stopping_threshold=0.001)])

trainer.train()
```

Figura 3.8: Codul ce surprinde antrenarea transformatorului XLM-RoBERTa-base

```
model = AutoModelForSequenceClassification.from_pretrained(
    "xlm-roberta-base",
    num_labels=num_classes,
    problem_type="multi_label_classification",
    id2label=id2label,
    label2id=label2id)
```

Figura 3.9: Definirea tipului de problemă

La fiecare pas de antrenare, modelul prezice ce emoție se regăsește în exemplul curent. Acest pas este crucial deoarece modelul învață unde a greșit prin intermediul funcției de loss, care compară predicția modelului cu răspunsul corect din setul de date și calculează eroarea.

Valoarea de 0.01 aplicată weight decay-ului este cea implicită, recomandată de Hugging Face pentru fine-tuning pe modele preantrenate. Este suficient de mică pentru a nu perturba cunoștințele acumulate de XLM-RoBERTa în preantrenare, dar suficientă pentru a preveni supraantrenarea pe datele proprii.

Rata de învățare aleasă este 2×10^{-5} , valoare situată în intervalul testat de autorii RoBERTa pentru fine-tuning pe taskuri de clasificare [12]. Am optat pentru aceasta deoarece o valoare mai mare ar risca să suprascrie cunoștințele pe care modelul le-a acumulat în preantrenare, fenomen cunoscut ca *catastrophic forgetting*.

Dimensiunea *batch-ului* a fost setată la 32 per GPU, ceea ce înseamnă că modelul a procesat câte 64 de exemple simultan. Pentru a evita antrenarea inutilă, am folosit mecanismul de early stopping cu `patience=2` și un număr maxim de 10 epoci.

Acest parametru oprește antrenarea dacă nu observă o îmbunătățire în două epoci consecutive, urmând ca mai apoi să salveze cel mai bun model înregistrat. Astfel, modelul a atins performanța maximă la epoca 3, iar antrenarea s-a oprit la epoca 5.

Antrenarea transformer-ului ales a fost formulată ca o problemă de clasificare multi-label, întrucât un text poate exprima în același timp tristețe și frică, sau bucurie și surpriză. Din acest motiv, funcția de loss *BCEWithLogitsLoss* calculează eroarea separat pentru fiecare din cele 19 clase. Prin această abordare am tratat fiecare emoție ca o decizie binară independentă, dacă este prezentă sau absentă.

La final, am aplicat funcția **sigmoid** pe ieșirile brute ale modelului (logits), pentru a produce un scor între 0 și 1 pentru fiecare emoție, fără ca acestea să se influențeze reciproc. Aceste scoruri reprezintă, în aplicația Deeplyn, intensitatea fiecărei emoții detectate în textul analizat, din care sunt extrase apoi primele trei emoții dominante.

3.5 Evaluarea modelului

Modelul antrenat a fost evaluat cu ajutorul mai multor metrici, ce surprind câte un aspect diferit al performanței. Cum problema implică clasificare multi-label pe 19 emoții cu frecvențe diferite în setul de date, am luat în considerare metricile de tip macro. În realitate, textele pot surprinde emoții diferite, chiar și în antiteză. Din acest motiv metricile descrise în următoarele paragrafe iau în considerare toate emoțiile și le tratează în mod egal și independent.

Prima metrică luată în calcul este **Macro F1**, aleasă tocmai pentru că distribuția emoțiilor din setul de antrenament nu este uniformă. Clase precum *neutral* sau *joy* apar de câteva ori mai frecvent decât *embarrassment* sau *guilt*, iar o metrică ponderată după frecvență ar oferi un scor global ridicat chiar dacă modelul performează slab pe clasele rare.

Macro F1 calculează scorul F1 independent pentru fiecare dintre cele 19 emoții și abia apoi face media, ceea ce înseamnă că o clasă rară contează la fel de mult ca una dominantă. În Deeplyn, unde modelul analizează notițele de jurnal și răspunsurile la quiz ale pacienților cu BPD, tocmai emoțiile mai puțin frecvente, precum vinovăția sau jena, pot fi cele mai semnificative pentru terapeut și pentru evoluția clinică a pacientului.

Putem calcula această metrică folosind următoarele formule:

$$\text{Macro F1} = \frac{1}{C} \sum_{c=1}^C F1_c \quad (3.2)$$

unde $C = 19$ reprezintă numărul de clase de emoții, iar $F1_c$ este scorul F1 calculat independent pentru fiecare clasă în parte, definit ca:

$$F1_c = \frac{2 \cdot P_c \cdot R_c}{P_c + R_c} \quad (3.3)$$

unde $P_c = \frac{TP_c}{TP_c + FP_c}$ reprezintă precizia, $R_c = \frac{TP_c}{TP_c + FN_c}$ reprezintă recall-ul pentru clasa c , iar TP_c , FP_c și FN_c sunt true positives, false positives și false negatives.

În Figura 3.10 putem observa o creștere constantă a scorului Macro F1 de la prima până la ultima și cea de-a treia epocă. Această creștere o putem traduce prin faptul că modelul a identificat emoțiile cu o acuratețe din ce în ce mai mare pe parcursul antrenării.

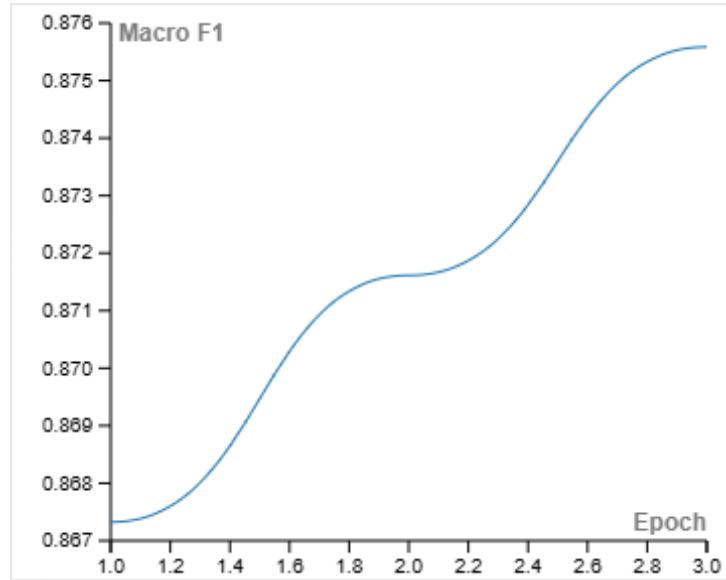


Figura 3.10: Evoluția scorului Macro F1 pe parcursul antrenării

Validarea modelului s-a realizat și prin urmărirea erorii pe setul de validare (*validation loss*). Acesta este, în practică, singurul indicator care arată dacă modelul generalizează sau doar memorează. În aplicația noastră, textele introduse de utilizatori sunt formulate liber, fără niciun tipar impus, așadar diferite față de datele pe care le-a învățat modelul. Prin monitorizarea acestei valori pentru fiecare epocă, am putut identifica momentul optim de oprire a antrenării. Deoarece antrenarea a fost formulată ca o problemă multi-label, eroarea a fost calculată prin funcția *BCEWithLogitsLoss*:

$$\mathcal{L} = -\frac{1}{N \cdot C} \sum_{i=1}^N \sum_{c=1}^C [y_{ic} \cdot \log \sigma(\hat{y}_{ic}) + (1 - y_{ic}) \cdot \log(1 - \sigma(\hat{y}_{ic}))] \quad (3.4)$$

unde N este numărul de exemple din setul de validare, $C = 19$ numărul de clase de emoții, $y_{ic} \in \{0, 1\}$ eticheta reală, $\hat{y}_{ic}$ logitul brut returnat de model, iar σ este funcția sigmoid aplicată asupra acestuia.

În Figura 3.11 se observă o ușoară creștere spre epoca a doua, urmată de o scădere semnificativă la a treia epocă. Această evoluție confirmă că modelul nu a intrat în supraantrenare (overfitting), așadar a continuat să generalizeze corect pe date nevăzute.

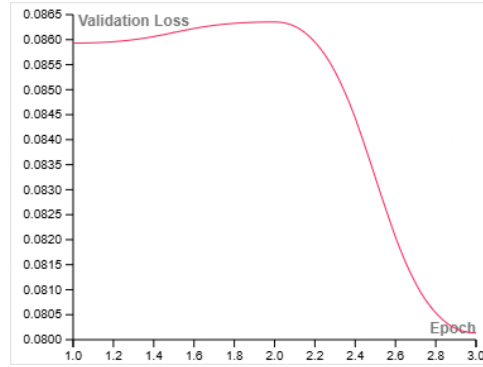


Figura 3.11: Evoluția scorului Validation Loss pe parcursul antrenării

Ultima metrică analizată este **Macro ROC AUC**. Aceasta măsoară capacitatea modelului de a distinge corect între prezența și absența unei emoții pentru fiecare din cele 19 clase, indiferent de pragul de decizie setat. Un scor de 1.0 reprezintă separare perfectă, iar 0.5 înseamnă că modelul ghicește aleatoriu. Putem calcula această metrică prin formula:

$$\text{Macro ROC AUC} = \frac{1}{C} \sum_{c=1}^C \text{AUC}_c \quad (3.5)$$

unde $C = 19$ este numărul de clase de emoții, iar AUC_c reprezintă aria de sub curba ROC pentru clasa c , definită ca:

$$\text{AUC}_c = \int_0^1 \text{TPR}_c(\text{FPR}_c) d(\text{FPR}_c) \quad (3.6)$$

unde $\text{TPR}_c = \frac{TP_c}{TP_c + FN_c}$ este rata true positive, iar $\text{FPR}_c = \frac{FP_c}{FP_c + TN_c}$ este rata false positive pentru clasa c .

Așa cum reiese din Figura 3.12, scorul a crescut progresiv pe parcursul celor trei epoci, ajungând la 0.992, o valoare care confirmă că modelul a învățat să recunoască cu precizie ridicată fiecare emoție în parte.

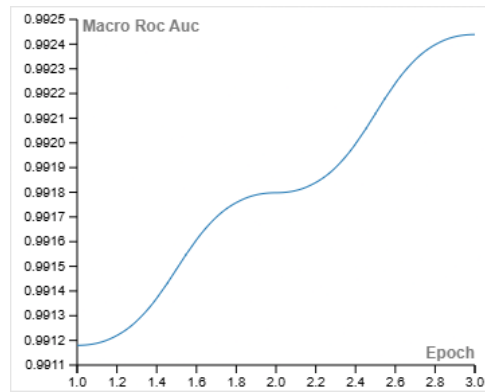


Figura 3.12: Evoluția scorului Macro ROC AUC pe parcursul antrenării

În ansamblu, cele trei metrice confirmă că modelul a învățat bine. Scorul Macro F1 și Macro ROC AUC au crescut constant, iar eroarea pe setul de validare a scăzut. Aceste rezultate demonstrează faptul că modelul nu a memorat datele de antrenare, ci a înțeles cum să recunoască emoțiile în texte pe care nu le-a mai văzut. Pentru o aplicație precum Deeplyn, unde fiecare utilizator scrie altfel, acest lucru contează în înțelegerea propriilor emoții și identificării lor.

Capitolul 4

Arhitectura platformei

4.1 Arhitectura generală a sistemului

Platforma Deeplyn adoptă o arhitectură de tip **client-server decuplat**, prin care frontend-ul și backend-ul comunică prin cereri **HTTP**, fiind două aplicații distincte ce rulează independent.

Abordarea propusă se diferențiază de o aplicație Django clasică bazată pe șabloane (*templates*). Dacă aceasta din urmă îi oferă serverului posibilitatea de a genera HTML direct către browser, în aplicația mea am lăsat această responsabilitate integral în seama frontend-ului dezvoltat în React.

Pentru partea de backend am folosit Django ca framework principal, alături de Django REST Framework (DRF), prin care toate răspunsurile către frontend sunt transmise în format **JSON**. Prin natura sa, Django propune modelul arhitectural **MVT** (*Model-View-Template*). Așa cum am menționat anterior, Deeplyn se bazează pe acest model doar parțial. Baza de date și logica întregii platforme sunt implementate în backend, în timp ce frontend-ul este responsabil de interfața vizuală.

Deeplyn se bazează totodată și pe o **arhitectură monolitică**. Am optat ca toate funcționalitățile și serviciile pe care aplicația mea le oferă să fie integrate în același spațiu, pentru ca fiecare aplicație Django să coopereze sub același nucleu. Am luat această decizie în stadiul incipient al dezvoltării platformei. Înainte de a stabili granițele clare dintre module, am ales acest model deoarece mi-a permis să iterez rapid și să înțeleg mult mai bine responsabilitatea fiecărei componente în parte.

Frontend-ul, implementat în React și Vite, este de tip **Single Page Application**. Browserul încarcă aplicația o singură dată, iar navigarea între ecrane este gestionată intern de React Router, care se asigură de gestionarea rutelor specifice pacienților, dar și ale terapeuților. Comunicarea client-server am asigurat-o prin **REST API** expus de Django REST framework. Pentru trimiterea cererilor HTTP și preluarea răspunsurilor JSON din frontend am folosit biblioteca **Axios**.

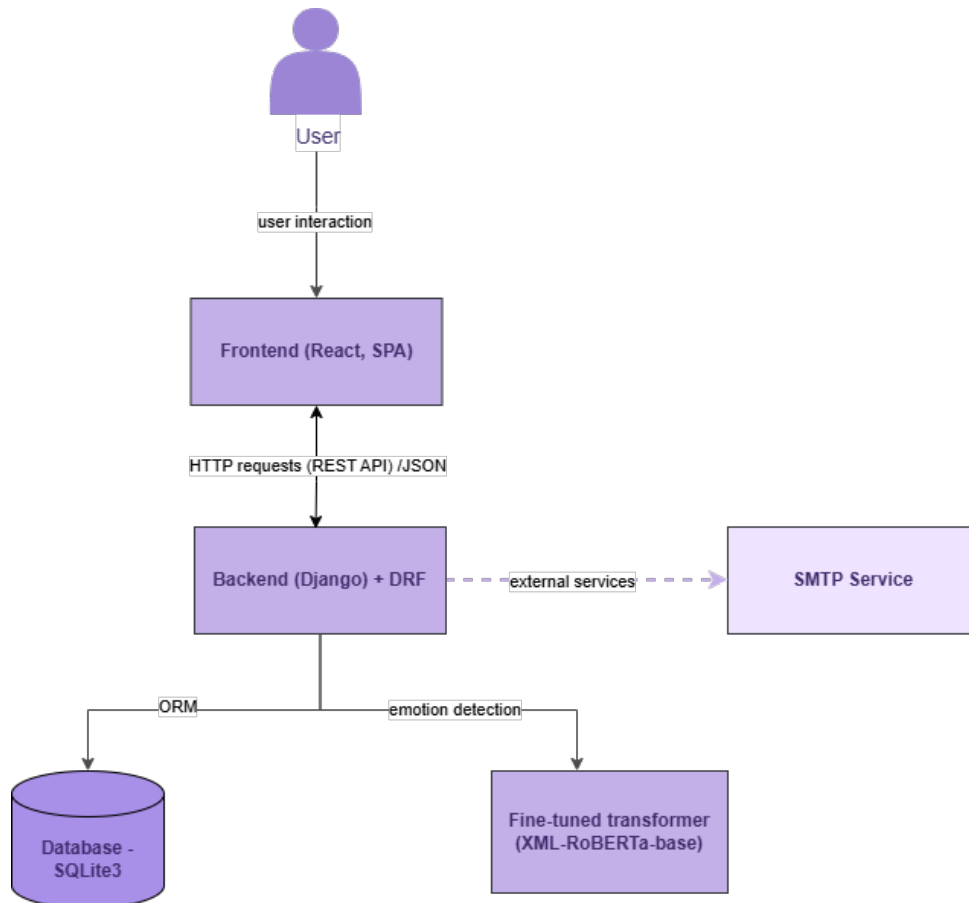


Figura 4.1: Arhitectura generală a sistemului Deeplyn

Deoarece frontend-ul și backend-ul rulează pe origini diferite, browserul blochează accesul la răspunsurile provenite dintr-o altă origine diferită de frontend. Am configurat **CORS** (*Cross-Origin Resource Sharing*) pe partea de backend pentru a permite cererile venite numai de la adresa frontend-ului.

De asemenea, am integrat un serviciu de e-mail bazat pe protocolul **SMTP**, folosit pentru confirmarea contului la înregistrare, recuperarea parolei și trimiterea notificărilor automate către utilizatori.

4.2 Structura bazei de date

Deoarece arhitectura backend-ului platformei mele este împărțită în aplicații Django, fiecare aplicație deține câte un fișier `models.py` unde sunt definite modelele corespunzătoare fiecărei funcționalități. Am implementat aceste modele prin ORM-ul oferit de Django și le-am stocat în SQLite.

Aplicația **users** stă la baza relațiilor dintre entitățile platformei, după cum reiese din Figura 4.2. Tabela omonimă extinde `AbstractUser` din Django. Pentru a identifica unic orice utilizator, am ales ca email-ul și username-ul să fie câmpuri unice, iar câmpul *role* separă clientul de terapeut. Terapeuții dețin un profil extins prin tabela **therapist**, conectată printr-un `OneToOneField`, unde sunt stocate datele trimise spre validare administratorului.

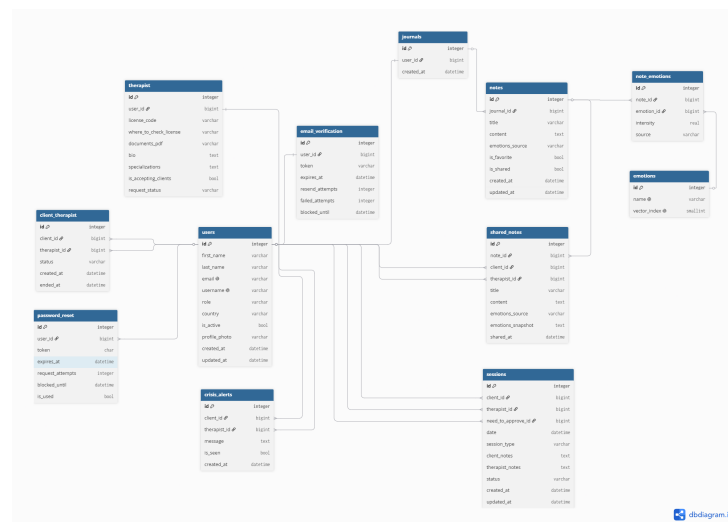


Figura 4.2: Diagrama relatională a aplicațiilor users, journal si therapy

Legătura dintre pacient și terapeut este surprinsă în tabela **client_therapist**, cu un câmp *status* ce poate lua una din stările *pending*, *active* și *ended*. La încheierea colaborării terapeutice, notițele partajate de pacient devin indisponibile pentru terapeut, iar sesiunile asociate se șterg automat.

Pentru a mă asigura că niciun atacator nu poate trimite un număr nelimitat de cereri, în tabelele **email_verification** și **password_reset** am adăugat câmpurile *attempts* și *blocked_until*. Am setat un maxim de 4 încercări, astfel încât la a 5-a tentativă eșuată, contul este blocat temporar până la expirarea intervalului stocat în *blocked_until*.

Modulul **journal** atribuie fiecărui utilizator un jurnal generat la crearea contului. În această componentă, utilizatorii își pot scrie gândurile în notițe proprii, fiecare

bela **sessions**. Programările pot fi inițiate de oricare dintre cele două părți, astfel că am inclus câmpul *need_to_approve* pentru a ști cine trebuie să confirme sesiunea propusă.

4.3 Interfața și implementarea platformei

În următoarea secțiune voi prezenta interfața utilizatorilor și scenariile de utilizare ale aplicației Deeplyn. Am încercat să construiesc un UI cât mai simplu și intuitiv, știind că utilizatorii trec prin momente dificile.

Prin termenul de "client" fac referință la utilizatorii obișnuiți ai platformei care suferă de tulburare de personalitate borderline sau care se confruntă cu dificultăți emoționale și caută un suport profesional.

Partea de frontend a fost gândită să ofere tuturor utilizatorilor o interfață intuitivă și ușor de utilizat. În proiectarea acesteia, am optat pentru o paletă cromatică simplă, în care predomină culorile alb, mov și lila. Alegerea acestor nuanțe se potrivește scopului aplicației, fiind frecvent asociate cu echilibrul emoțional și introspecția.

4.3.1 Register și Login

Prima pagină de intrare în aplicație o reprezintă LandingPage-ul, unde utilizatorii au acces la o scurtă descriere a platformei și au opțiunea de a se înregistra fie ca utilizator obișnuit (client/pacient), fie ca terapeut.

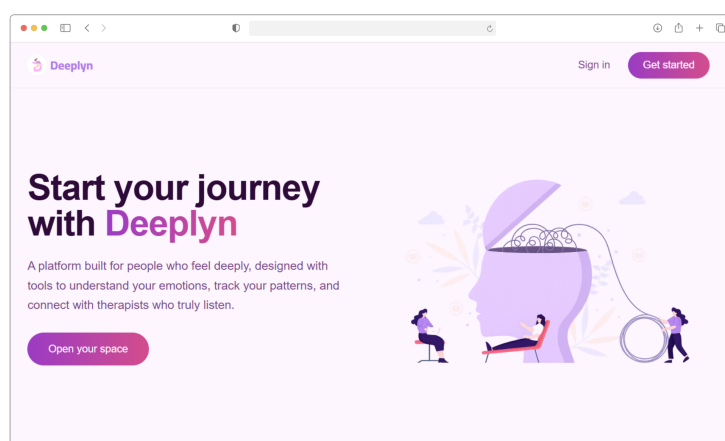


Figura 4.4: Captura de ecran a paginii Landing Page

Paginile de înregistrare prezentate în figura4.5 și figura4.6 includ câmpurile necesare pentru crearea unui cont nou, atât din perspectiva utilizatorilor cu BPD cât și

din perspectiva terapeuților. Alături de formularul de înregistrare, am ales să integrez o ilustrație distinctă pentru utilizatorii obișnuiți, respectiv terapeuți, cu scopul de a evidenția rolurile specifice pe care aceștia le vor avea în cadrul aplicației.

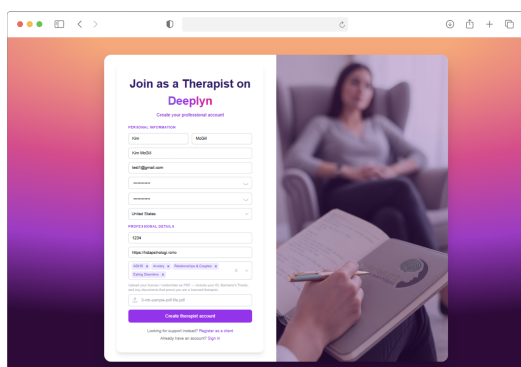


Figura 4.5: Captura de ecran a paginii de înregistrare pentru terapeuți specializați

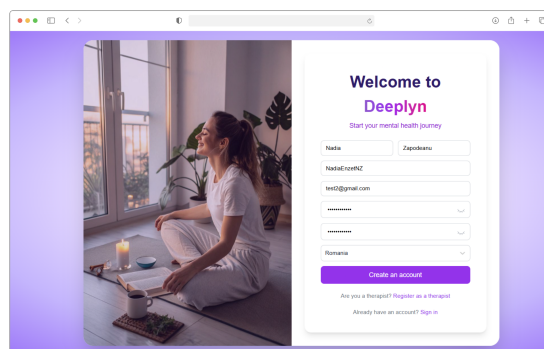


Figura 4.6: Captura de ecran a paginii de înregistrare pentru utilizatorii obișnuiți/pacienți

Pentru o autentificare completă, ambii utilizatori sunt nevoiți să își valideze adresa de email. De îndată ce formularul de înregistrare este trimis, serviciul de email le oferă utilizatorilor un token de 6 caractere alfanumerice generat aleatoriu pe adresa de email introdusă. Este necesară introducerea aceluiași token în pagina VerifyEmail(figura 4.7) pentru a fi activat contul.

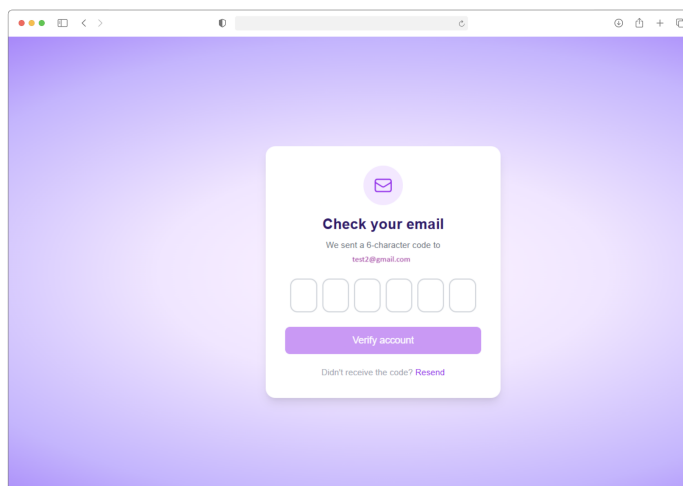


Figura 4.7: Captura de ecran a paginii VerifyEmail

În cazul înregistrării ca terapeut, este necesară validarea contului de către un administrator. Această abordare a fost inspirată din bunele practici ale platformelor profesionale existente în domeniu, unde verificarea manuală a informațiilor despre terapeut reprezintă o garanție suplimentară pentru siguranța pacienților.

Utilizatorii ce au un cont activ pot interacționa cu aplicația prin intermediul paginii de Login (figura 4.8). Aceeași pagină este responsabilă atât pentru logarea utilizatorilor obișnuiți cât și pentru logarea terapeuților. Userii introduc emailul sau usernameul și parola pentru a se autentifica. Dacă informațiile date de ei sunt corecte, platforma îi direcționează către pagina de Dashboard, unde au acces la toate funcționalitățile aplicației.

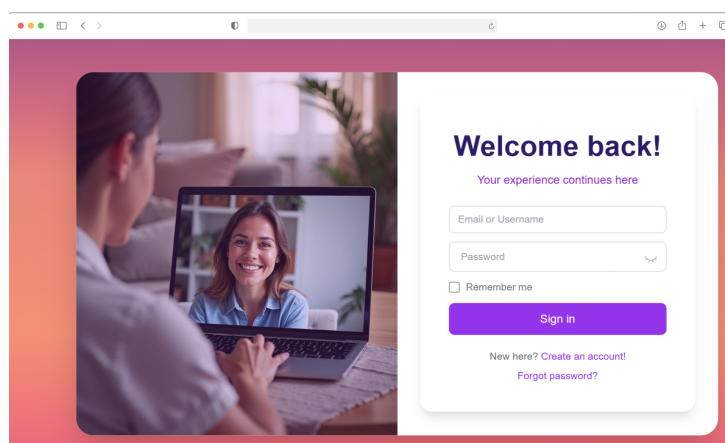


Figura 4.8: Captura de ecran a paginii de Login

4.3.2 Dashboard

Interfața Dashboard-ului reprezintă pagina principală a platformei Deeplyn și oferă utilizatorilor o imagine de ansamblu completă asupra activității lor zilnice.

Din perspectiva pacienților, pagina le oferă o imagine centralizată asupra activităților lor pe platformă (figura 4.9). Am asigurat accesul direct către funcționalitățile aplicației prin patru carduri navigabile, acestea fiind Daily Quiz, Therapist, Journal și Exercises. Imediat sub acestea, sunt afișate patru statistici de activitate, streak-ul zilnic curent, numărul de notițe de jurnal, chestionarele completate din secțiunea de Daily Quiz și exercițiile efectuate.

Secțiunea inferioară a paginii include un grafic al evoluției emoționale, generat pe baza emoțiilor preluate din toate cele trei module: **Journal**, **DailyQuiz** și **Exercises**. Am adăugat și o componentă vizuală cu detaliile următoarei sesiune cu terapeutul, vizibilă utilizatorilor ce au o relație activă cu acesta.

Pentru utilizatorii cu rol de terapeut, Dashboard-ul este structurat în jurul gestionării pacienților. Această pagină este compusă din patru carduri de statistici, ce

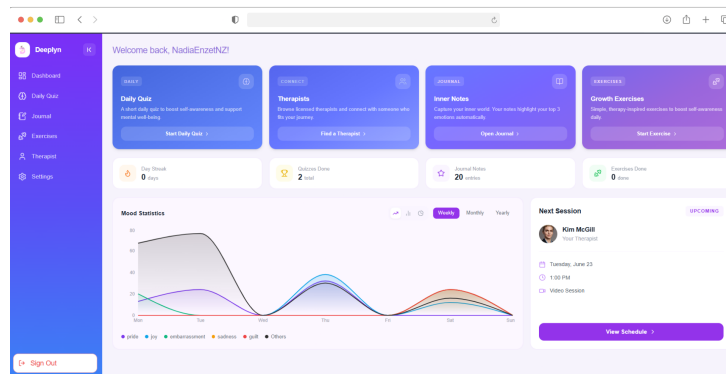


Figura 4.9: Captura de ecran a paginii Dashboard - perspectiva utilizatorilor cu BPD

sintetizează numărul de pacienți activi la momentul actual, cererile în așteptare de la pacienții noi, sesiunile de terapie din săptămâna curentă și notițele de jurnal distribuite ale utilizatorilor (figura 4.10).

Fiecare card este interactiv și redirecționează terapeutul specializat către pagina corespunzătoare elementului ales. Am inclus de asemenea o listă cronologică a ședințelor viitoare, fiind o componentă navigabilă. În panoul lateral din dreapta sunt afișate cererile noi de conexiune, activitatea recentă a pacienților și chestionarele create de terapeut pentru fiecare user.

Ambele interfețe includ un panou lateral de navigare (sidebar), care oferă acces rapid la toate funcționalitățile disponibile, adaptate rolului fiecărui utilizator în parte.

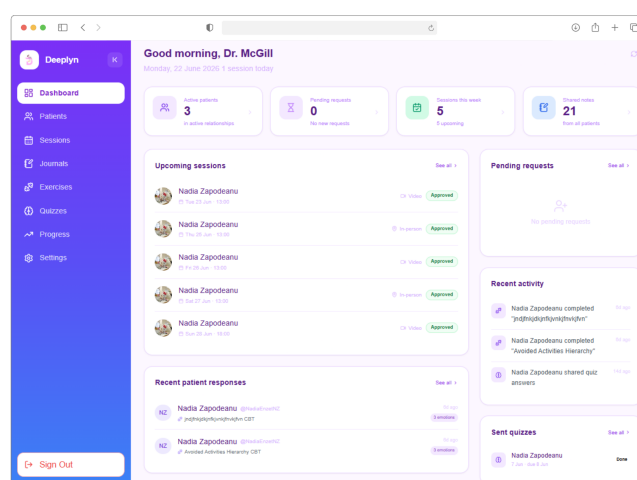


Figura 4.10: Captura de ecran a paginii Dashboard - perspectiva terapeutului specializat

4.3.3 Journal

Pagina Journal este accesibilă tuturor utilizatorilor autentificați pe platformă. Utilizatorul obișnuit o poate accesa indiferent dacă are sau nu un terapeut, având la dispoziție un spațiu dedicat pentru a-și nota gândurile și trăirile în mod liber. Am decis ca această secțiune să aibă același UI în ambele cazuri, atât pentru pacient cât și pentru terapeut, după cum se poate observa în figurile 4.11 și 4.12.

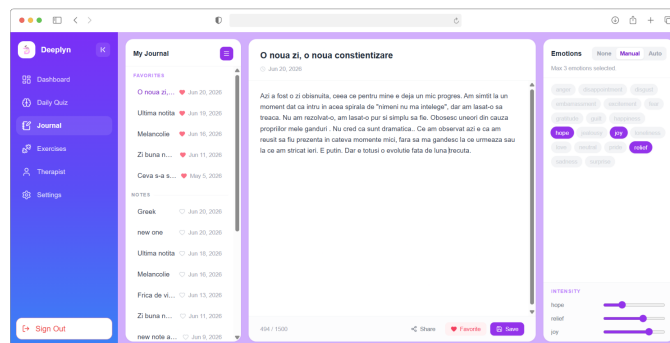


Figura 4.11: Captura de ecran a paginii Journal din perspectiva utilizatorilor cu BPD

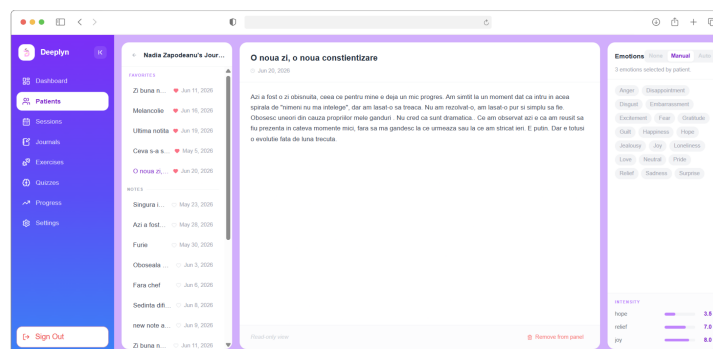


Figura 4.12: Captura de ecran a paginii Journal din perspectiva terapeutului

Asemănarea dintre cele două pagini pune accent pe ideea unui jurnal real, astfel că ambele tipuri de utilizatori se raportează la aceeași structură a panoului, la aceleași notițe și scoruri de intensitate emoțională. Diferența dintre ele constă strict în conținut. Terapeutul poate observa doar însemnările distribuite de pacienții săi, în timp ce pacientul interacționează direct cu cele proprii. Am urmărit astfel să creez un spațiu în care accesul la datele personale funcționează strict pe bază de consimțământ.

În interiorul acestei pagini regăsim trei componente principale prin care utilizatorii pot interacționa cu datele lor. Partea stângă afișează lista de notițe, fiecare fiind identificată prin titlu, data creării și starea ei (favorită sau normală).

La accesarea unei notițe din listă, aceasta se deschide în zona centrală pentru a putea fi vizualizată. Pentru a crea o notiță nouă, utilizatorii trebuie să introducă un titlu și conținutul propriu-zis, care este limitat la maximum 1500 de caractere. Etichetarea se gestionează din panoul din dreapta prin trei opțiuni, anume:

- **None:** utilizatorii pot alege să nu își eticheteze cu emoții notița creată.
- **Manual:** utilizatorul își alege obligatoriu trei emoții principale, împreună cu trei intensități ale acestora.
- **Auto:** transformatorul XLM-RoBERTa-base citește întreg conținutul și returnează cele trei emoții predominante.

Odată ce toate aceste câmpuri sunt completate, notița este salvată în jurnal. Ulterior, ea poate fi ștearsă, modificată direct în interiorul ei, partajată cu terapeutul sau marcată ca fiind favorită.

Din perspectiva terapeutului, acesta are permisiunea doar de a vizualiza notițele distribuite de un anumit pacient și de a le șterge din panoul său. Ștergerea notițelor în acest caz elimină înregistrarea din tabela **shared_notes**, notița originală a pacientului rămânând intactă în jurnalul său.

4.3.4 DailyQuiz

Pagina dedicată modulului **quiz** încurajează utilizatorii să practice introspecția și conștientizarea propriilor trăiri în fiecare zi. Pentru userii ce folosesc independent platforma, am optat să adaug 500 de întrebări default extrase direct din terapia cognitiv-comportamentală și terapia dialectic-comportamentală. Pentru a asigura diversitatea, întrebările sunt selectate aleatoriu, diferite de la utilizator la utilizator, și nu se repetă până când nu este epuizat întregul set de întrebări.

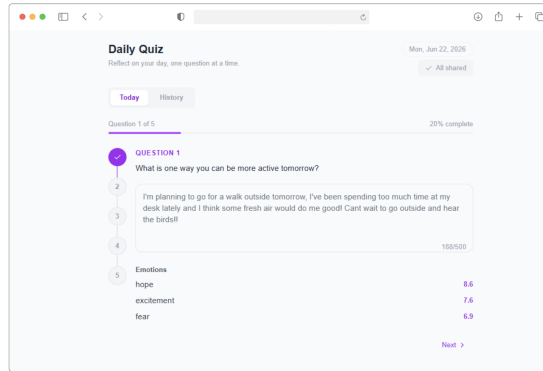


Figura 4.13: Captura de ecran a paginii de DailyQuiz pentru pacienții cu BPD

Din punct de vedere tehnic, logica din spatele acestei funcționalități urmează un flux bine definit în baza de date:

- Verificarea unui quiz activ de la terapeut: sistemul caută mai întâi dacă există un chestionar activ alocat de terapeut pentru ziua curentă. Dacă acesta este găsit și nu a fost încă finalizat de către utilizator, este returnat direct în interfață.
- Verificarea quiz-ului automat existent: În cazul în care nu există un quiz trimis de terapeut, aplicația verifică dacă a fost deja generat un chestionar automat (de tip APP) pentru utilizatorul respectiv în ziua curentă. Dacă acesta există în baza de date, va fi afișat în interfață.
- Generarea unui quiz nou: Dacă nu se găsește nicio înregistrare validă pentru ziua respectivă, este apelată funcția de generare. Serverul preia toate ID-urile celor 500 de întrebări predefinite și elimină din listă ID-urile întrebărilor deja completate în chestionarele anterioare ale utilizatorului. Din restul rămas, se extrag aleatoriu 5 întrebări. Acestea sunt salvate în baza de date, fiind apoi trimise către frontend.
- Resetarea ciclului de întrebări se produce în momentul în care în lista de întrebări disponibile rămân mai puțin de 5 opțiuni. În acest punct, istoricul întrebărilor completate este ignorat, iar funcția extrage din nou 5 întrebări random din întregul set de 500 de întrebări.

Terapeutul are posibilitatea de a crea întrebări și chestionare zilnice, adaptate la nevoile pacienților săi. Fișa de autoreflexie poate conține între 1 și 20 de întrebări, care

fie sunt redactate direct de specialist, fie sunt selectate din setul de întrebări predefinite ale aplicației, conform ilustrației de mai jos (figura 4.14).

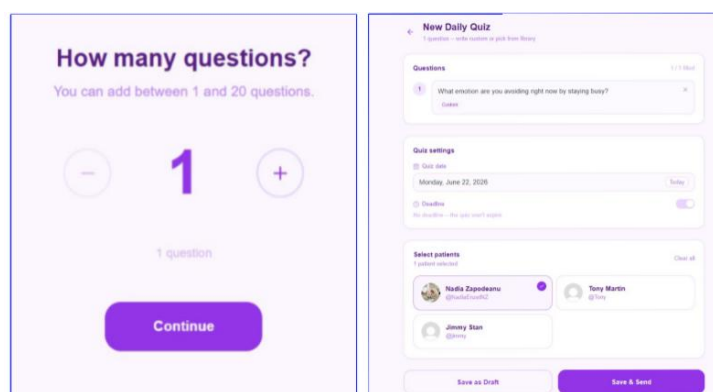


Figura 4.14: Capturi de ecran a componentelor paginii *Quizzes* - perspectiva terapeutului specializat

Setul de acțiuni disponibile terapeutului este mai extins față de cel al unui utilizator obișnuit. Terapeutul are posibilitatea de a crea chestionare, le poate atribui pacienților, le poate păstra ca draft sau le poate elimina din panoul său. Singura limitare este că întrebările predefinite ale aplicației nu pot fi șterse. Toate aceste funcționalități sunt ilustrate în figura 4.15.

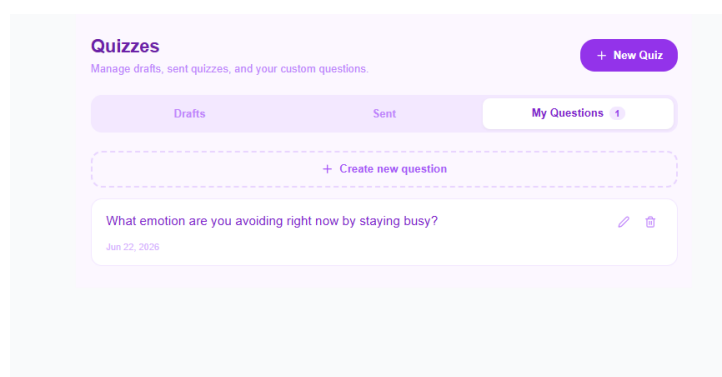


Figura 4.15: Captura de ecran a componentelor paginii *Quizzes* - acțiuni disponibile terapeutului

4.3.5 Exercises

Exercițiile din Deeplyn sunt disponibile tuturor utilizatorilor, indiferent dacă au sau nu un terapeut activ pe platformă. Conținutul este structurat pe baza tehnicilor DBT și CBT discutate în capitolele anterioare.

Pacienții care au o relație activă cu un terapeut beneficiază în plus de exercițiile personalizate create de acesta, similar cu modul în care funcționează quiz-urile. Diferența față de alte module este că un exercițiu poate fi completat de mai multe ori, fiecare completare fiind salvată separat în istoric. Am hotărât această decizie de design deoarece exersarea skill-urilor capătă sens odată ce sunt repetate.

La finalul unui exercițiu, utilizatorul poate alege să împărtășească exercițiile completate cu terapeutul său, cu posibilitatea de a adăuga un comentariu opțional. Ca și în jurnal sau în quiz-uri, etichetarea exercițiilor cu emoții este prezentă și aici (figura 4.16 și figura 4.17).

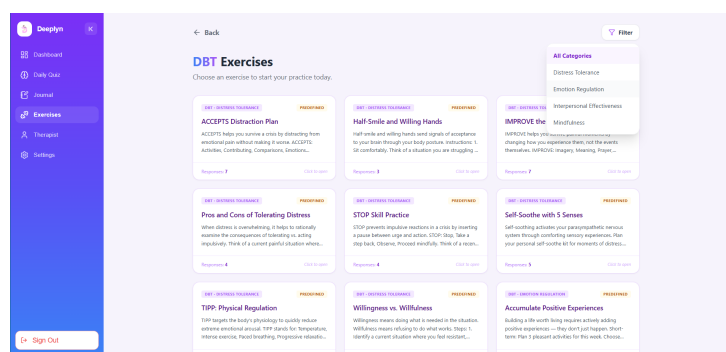


Figura 4.16: Captura de ecran a paginii Exercises (pacienti), setul de exerciții predefinite

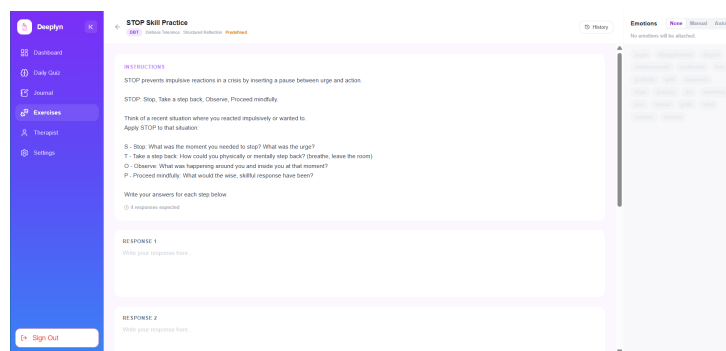


Figura 4.17: Captura de ecran a paginii Exercises (pacienti), exemplu de exercițiu

Specialistul are aceleași permisiuni ca în modulul de quiz-uri. În figura 4.18 putem observa pagina dedicată modulului alături de cele cinci butoane care îi oferă posibilitatea de a naviga între exercițiile predefinite din aplicație (*Exercise from App*), exercițiile proprii create anterior (*My Exercises*), exercițiile sale nedistribuite și salvate ca draft (*Drafts*), un formular de creare a unui exercițiu nou (*Create*), precum și exercițiile completate și distribuite de pacienții săi (*Patients Exercises*).

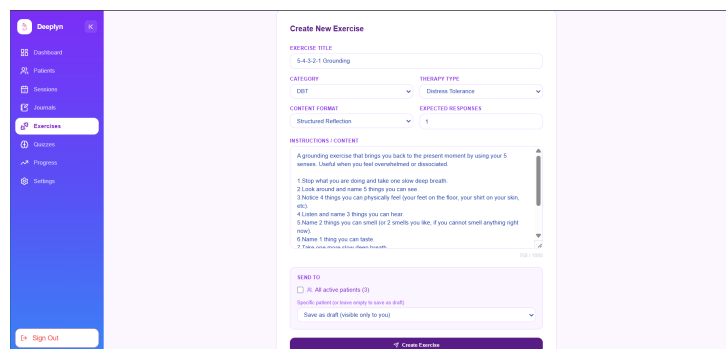


Figura 4.18: Captura de ecran a paginii *Exercises* - crearea unui exercițiu pentru pacienți

4.3.6 Therapist

În cadrul acestei pagini, fiecare utilizator are posibilitatea de a explora lista de specialiști disponibili prin intermediul unor filtre care sortează terapeuții în funcție de țara lor de activitate și de specializările deținute. Pentru a face cât mai plăcută experiența utilizatorului, pagina este concepută să afișeze doar acei terapeuți care acceptă pacienți noi. Detaliile fiecărui specialist sunt evidențiate printr-o scurtă descriere și prin prezentarea specializărilor sale. Aceste date sunt preluate din baza de date prin intermediul câmpurilor *bio* și *specialization* ale tabelii *therapist* (figura 4.19).

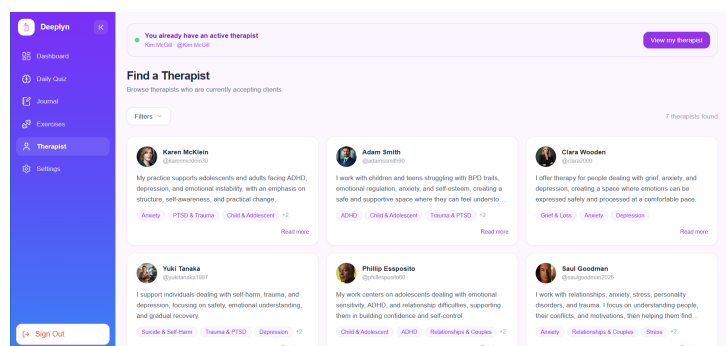


Figura 4.19: Captura de ecran a paginii *Therapist*

Obiectivul pe care l-am urmărit în implementarea acestei pagini este de a oferi ocazia tuturor utilizatorilor să își creeze o conexiune stabilă cu un terapeut specializat. Pentru ca aceștia să contacteze un specialist, pot apăsa pe butonul **Connect**. În acest mod se trimite automat un e-mail terapeutului pentru a accepta sau respinge cererea pacientului.

În momentul în care o relație de tip pacient-terapeut devine activă, interfața paginii se modifică dinamic și afișează acest status. Astfel, în secțiunea superioară a ecranului este integrată o componentă de navigare rapidă către funcționalitățile esențiale.

Din această zonă, utilizatorul poate accesa pagina cu detaliile extinse ale terapeutului, un calendar interactiv conceput pentru inițierea, acceptarea sau respingerea sesiunilor de terapie, o secțiune cu notițele distribuite cu terapeutul, precum și un istoric al exercițiilor recente (figura 4.20).

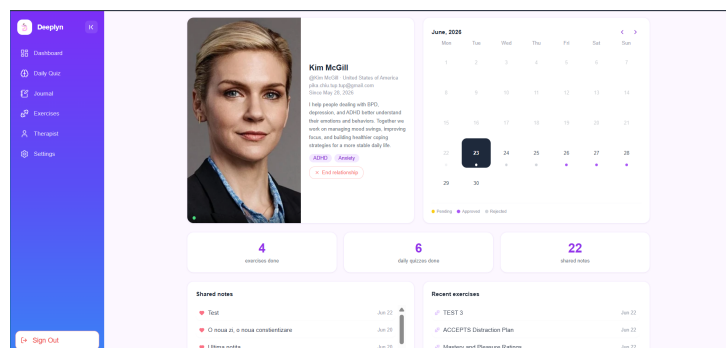


Figura 4.20: Captura de ecran a paginii *My Therapist*

Din punct de vedere etic și ca o bună practică în dezvoltarea soluțiilor software dedicate sănătății mintale, am pus la dispoziția userilor un buton special pentru managementul situațiilor de criză, precum se poate observa în figura 4.20. Pentru momentele în care utilizatorul se simte copleșit, acesta poate apela la terapeutul său prin intermediul butonului, ce îi trimite o notificare pe e-mail.

Această pagină este accesibilă exclusiv terapeutului și afișează datele emoționale ale tuturor pacienților săi. Utilizatorul poate monitoriza starea fiecărui pacient în funcție de informațiile transmise din toate sursele distribuite de pacient.

Pentru o analiză detaliată, datele pot fi filtrate pe diferite intervale de timp (săptămânal, lunar sau anual) și pot fi vizualizate sub trei forme grafice, anume sub forma unei diagrame liniare, grafic de tip plăcintă sau diagramă cu bare stivuite.

În această secțiune vor fi afișate cele mai predominante emoții (în limita a maximum cinci), urmate de categoria *Others* (figura 4.21).

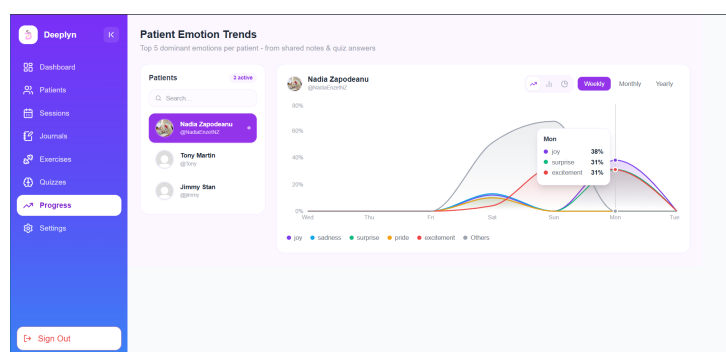


Figura 4.21: Captura de ecran a paginii *Progress*

Capitolul 5

Direcții viitoare de dezvoltare

Platforma Deeplyn, în forma sa actuală, răspunde nevoilor esențiale ale utilizatorilor săi. Cu toate acestea, o platformă dedicată sănătății mintale poate evolua continuu, astfel că, în cele ce urmează, am conturat câteva direcții în care îmi propun să o dezvolt în continuare.

O nouă funcționalitate ar fi integrarea unei componente dedicate comunicării directe între terapeut și pacient, în timp real, fără a părăsi platforma. În starea prezentă, singurul mijloc de comunicare în afara sesiunilor rămâne emailul, însă o implementare viitoare ar presupune folosirea WebSocket-urilor prin Django Channels, ceea ce ar permite schimbul de mesaje instant, fără reîncărcarea paginii.

O altă direcție de dezvoltare este introducerea unei rubrici de feedback, ce ar permite utilizatorilor să își exprime părerile și experiențele legate de exercițiile parcurse, să propună tehnici noi sau să împărtășească ce i-a ajutat. Utilizatorii ar putea alege dacă doresc să posteze anonim sau cu numele lor. Consider că o astfel de rubrică ar aduce un plus-valoare platformei, unde utilizatorii contribuie activ la îmbunătățirea conținutului.

O altă implementare pe care mi-o propun este integrarea unui chatbot în platformă, capabil să poarte o conversație empatică cu utilizatorul și să îi recomande exerciții potrivite în funcție de ceea ce descrie. Utilizatorul ar putea scrie liber despre cum se simte, iar chatbot-ul ar analiza inputul și ar sugera unul sau mai multe exerciții din baza de date Deeplyn, relevante pentru starea sa din acel moment. Acest lucru ar fi deosebit de util pentru utilizatorii care nu au încă un terapeut, adăugând platformei un asistent digital activ, disponibil oricând este nevoie de îndrumare.

Capitolul 6

Utilizarea uneltelor de Inteligență Artificială

Pe parcursul dezvoltării platformei Deeplyn, am recurs la câteva instrumente de inteligență artificială (LLM) în mai multe etape ale proiectului.

Antrenarea transformatoarelor a fost, cu siguranță, zona unde sprijinul extern a contat cel mai mult. Nu aveam experiență anterioară cu evaluarea modelelor NLP. Astfel, am folosit aceste resurse ca punct de plecare pentru a înțelege ce măsoară Macro F1, ROC AUC și Validation Loss și de ce fiecare dintre ele este relevantă într-un context de clasificare multi-label. Totodată, aceste metrice au fost validate ulterior și de profesoara mea coordonatoare.

De asemenea, am folosit aceste unelte în identificarea erorilor din cod, atunci când comportamentul aplicației nu era cel așteptat. Tot cu ajutorul lor am generat o parte din exercițiile predefinite din platformă, bazate pe tehnicile DBT și CBT, pe care le-am verificat și adaptat ulterior.

Nu în ultimul rând, aceste unelte m-au ajutat în corectarea diacriticelor și a greșelilor gramaticale din redactarea acestei lucrări.

Concluzii

Aplicația prezentată în această lucrare a urmărit să ofere persoanelor care se confruntă cu tulburarea de personalitate borderline un spațiu digital în care să își gestioneze emoțiile și simptomele, fie independent, fie alături de un terapeut din platformă.

Prin intermediul platformei Deeplyn, am dorit să aduc o completare a psihoterapiei clasice, punând la dispoziție instrumentele necesare inspirate din cadrul sesiunilor terapeutice. Într-o perioadă în care tehnologia face parte din viața de zi cu zi, o astfel de platformă reduce distanța dintre utilizator și resursele de care are nevoie.

Realizarea acestei lucrări a reprezentat atât o provocare tehnică, cât și o experiență cu o valoare personală aparte, deoarece mi-a oferit posibilitatea de a îmbina cunoștințele acumulate în anii de studiu cu un domeniu diferit, dar care mă pasionează.

Bibliografie

- [1] A. P. Association, *Diagnostic and Statistical Manual of Mental Disorders*. American Psychiatric Association Publishing, Mar. 2022, ISBN: 0890425752. DOI: 10.1176/appi.books.9780890425787. adresa: <http://dx.doi.org/10.1176/appi.books.9780890425787>.
- [2] *React Documentation*, <https://react.dev/learn>.
- [3] *Stack Overflow Developer Survey 2025*, <https://survey.stackoverflow.co/2025>, 2025.
- [4] Django Software Foundation, *Django Documentation*, 2026. adresa: <https://docs.djangoproject.com/>.
- [5] V. Flasbeck, S. Popkirov, A. Ebert și M. Brüne, „Altered interoception in patients with borderline personality disorder: a study using heartbeat-evoked potentials,” *Borderline Personality Disorder and Emotion Dysregulation*, vol. 7, nr. 1, Oct. 2020, ISSN: 2051-6673. DOI: 10.1186/s40479-020-00139-1. adresa: <http://dx.doi.org/10.1186/s40479-020-00139-1>.
- [6] A. Vaswani et al., „Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [7] D. Demszky, D. Movshovitz-Attias, J. Ko, A. Cowen, G. Nemade și S. Ravi, „Go-Emotions: A Dataset of Fine-Grained Emotions,” în *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, Dataset accesat: https://huggingface.co/SamLowe/roberta-base-go_emotions, Association for Computational Linguistics, 2020, pp. 4040–4054. DOI: 10.18653/v1/2020.acl-main.372. adresa: <http://dx.doi.org/10.18653/v1/2020.acl-main.372>.

- [8] K. R. Scherer și H. G. Wallbott, „Evidence for universality and cultural variation of differential emotion response patterning,” *Journal of Personality and Social Psychology*, vol. 66, nr. 2, pp. 310–328, 1994, Dataset disponibil la: <https://huggingface.co/datasets/savalera/isear-from-original>, ISSN: 0022-3514. DOI: 10.1037/0022-3514.66.2.310. adresa: <http://dx.doi.org/10.1037/0022-3514.66.2.310>.
- [9] H. Rashkin, E. M. Smith, M. Li și Y.-L. Boureau, „Towards Empathetic Open-domain Conversation Models: A New Benchmark and Dataset,” în *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, Dataset accesat <https://huggingface.co/datasets/Bdotloh/empathetic-dialogues-contexts>, Association for Computational Linguistics, 2019, pp. 5370–5381. DOI: 10.18653/v1/p19-1534. adresa: <http://dx.doi.org/10.18653/v1/P19-1534>.
- [10] E. Saravia, H.-C. T. Liu, Y.-H. Huang, J. Wu și Y.-S. Chen, „CARER: Contextualized Affect Representations for Emotion Recognition,” în *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Dataset accesat <https://huggingface.co/datasets/dair-ai/emotion>, Association for Computational Linguistics, 2018, pp. 3687–3697. DOI: 10.18653/v1/d18-1404. adresa: <http://dx.doi.org/10.18653/v1/D18-1404>.
- [11] S. Pulle, *20-Emotion Text Classification Dataset*, HuggingFace, 2025. adresa: <https://huggingface.co/datasets/shreyaspulle98/emotion-dataset-20-emotions>.
- [12] Y. Liu et al., „Roberta: A robustly optimized bert pretraining approach,” *arXiv preprint arXiv:1907.11692*, 2019.